

CUDA Chapter 3 Notes: Multidimensional Grids, Data Mapping, Image Blur, and Matrix Multiplication

1 Multidimensional Grid Organization

Chapter 3 extends the one-dimensional thread mapping from vector addition to two- and three-dimensional data. The basic CUDA hierarchy remains

Kernel launch $\rightarrow$ Grid $\rightarrow$ Blocks $\rightarrow$ Threads.

A grid and a block may each have up to three dimensions. CUDA provides four built-in variables for describing this hierarchy:

Variable	Meaning
<code>gridDim</code>	grid dimensions, measured in blocks
<code>blockDim</code>	block dimensions, measured in threads
<code>blockIdx</code>	index of the current block
<code>threadIdx</code>	index of the current thread inside its block

Each variable has **x**, **y**, and **z** components. For example,

```
1 dim3 dimGrid(2, 2, 1);  
2 dim3 dimBlock(4, 2, 2);  
3  
4 KernelFunction<<<dimGrid, dimBlock>>>(...);
```

creates

$$2 \times 2 \times 1 = 4$$

blocks, with

$$4 \times 2 \times 2 = 16$$

threads in each block, for a total of

$$4 \times 16 = 64$$

threads.

Conceptually, a multidimensional grid is simply a multidimensional array of blocks, while each block is a multidimensional array of threads. The dimensions are organizational coordinates: they make it convenient to map threads to naturally multidimensional data such as images, matrices, and volumes.

The total number of threads in one block must satisfy the hardware limit. A common CUDA limit is

$$\text{blockDim.x} \times \text{blockDim.y} \times \text{blockDim.z} \leq 1024.$$

Unused dimensions are set to 1.

1.1 Two-dimensional thread coordinates

Images and matrices are naturally two-dimensional, so a two-dimensional grid of two-dimensional blocks is a convenient mapping.

For a thread processing a two-dimensional data set,

```
1 int col = blockIdx.x * blockDim.x + threadIdx.x;  
2 int row = blockIdx.y * blockDim.y + threadIdx.y;
```

The corresponding coordinates are

$$\begin{aligned} col &= \text{blockIdx.x} \times \text{blockDim.x} + \text{threadIdx.x}, \\ row &= \text{blockIdx.y} \times \text{blockDim.y} + \text{threadIdx.y}. \end{aligned}$$

Thus, the x -direction usually maps to columns and the y -direction maps to rows. For example, with 16×16 blocks, a thread with

$$\text{blockIdx} = (0, 1), \quad \text{threadIdx} = (0, 0)$$

has

$$col = 0, \quad row = 16,$$

so it is responsible for the data element at $(row, col) = (16, 0)$.

2 Mapping Multidimensional Data to Linear Memory

Although an image or matrix is conceptually two-dimensional, memory is addressed linearly. In C and C++, ordinary multidimensional arrays use **row-major order**: all elements of one row are stored consecutively before the next row begins.

For a two-dimensional array with `width` columns,

$$idx = row \times width + col$$

maps the two-dimensional coordinate (row, col) to a one-dimensional offset.

Consider the 4×4 matrix

$$\begin{bmatrix} M_{0,0} & M_{0,1} & M_{0,2} & M_{0,3} \\ M_{1,0} & M_{1,1} & M_{1,2} & M_{1,3} \\ M_{2,0} & M_{2,1} & M_{2,2} & M_{2,3} \\ M_{3,0} & M_{3,1} & M_{3,2} & M_{3,3} \end{bmatrix}.$$

In row-major memory it is stored as

$$\begin{aligned} &M_{0,0}, M_{0,1}, M_{0,2}, M_{0,3}, \\ &M_{1,0}, M_{1,1}, M_{1,2}, M_{1,3}, \\ &M_{2,0}, M_{2,1}, M_{2,2}, M_{2,3}, \\ &M_{3,0}, M_{3,1}, M_{3,2}, M_{3,3}. \end{aligned}$$

For example,

$$M_{2,1} \longrightarrow 2 \times 4 + 1 = 9,$$

so $M_{2,1}$ is stored at `M[9]`.

This same conversion is used repeatedly in CUDA kernels:

```
1 int idx = row * width + col;
```

3 Covering a Two-dimensional Image with Blocks

Suppose an image has

$$width = 76, \quad height = 62,$$

and each block contains

$$16 \times 16$$

threads.

The required grid dimensions are

$$N_x = \left\lceil \frac{76}{16} \right\rceil = 5, \quad N_y = \left\lceil \frac{62}{16} \right\rceil = 4.$$

Therefore, the kernel launch uses a 5×4 grid of blocks:

```
1 dim3 dimBlock(16, 16);
2 dim3 dimGrid((76 + 15) / 16,
3             (62 + 15) / 16);
```

The logical thread region covers

$$80 \times 64$$

positions, while the real image contains only

$$76 \times 62$$

pixels.

This produces four types of block regions:

Region	Valid part of the block	Valid threads
Interior	fully inside the image	$16 \times 16 = 256$
Right boundary	only 12 valid columns	$12 \times 16 = 192$
Bottom boundary	only 14 valid rows	$16 \times 14 = 224$
Lower-right corner	12 columns and 14 rows	$12 \times 14 = 168$

The interior blocks are completely covered by valid image pixels. The blocks on the right, bottom, and lower-right corner contain some threads whose coordinates fall outside the image.

Therefore, every image-processing kernel needs the two-dimensional boundary condition

```
1 if (col < width && row < height) {
2     // process pixel(row, col)
3 }
```

This is the two-dimensional counterpart of the one-dimensional check `if (i < n)`.

4 Image Blur

4.1 Basic idea

A simple image blur replaces each output pixel by the average of a local neighborhood around the corresponding input pixel.

For a 3×3 blur centered at (row, col) ,

$$P_{\text{out}}(\text{row}, \text{col}) = \frac{1}{N_{\text{valid}}} \sum_{\text{valid neighbors}} P_{\text{in}}(r, c).$$

For an interior pixel such as (25, 50), the 3×3 neighborhood is

$$\begin{aligned} & (24, 49), (24, 50), (24, 51), \\ & (25, 49), (25, 50), (25, 51), \\ & (26, 49), (26, 50), (26, 51). \end{aligned}$$

The thread responsible for output pixel (25, 50) reads these nine values, adds them, divides by 9, and writes one blurred output pixel.

If BLUR_SIZE denotes the radius of the neighborhood, the patch width is

$$2 \times \text{BLUR_SIZE} + 1.$$

For example,

BLUR_SIZE	Patch size
1	3×3
2	5×5
3	7×7

The number of candidate input pixels examined by an interior thread is

$$(2 \times \text{BLUR_SIZE} + 1)^2.$$

4.2 Core blur kernel

One thread still computes one output pixel. The main difference from grayscale conversion is that the thread must loop over neighboring input pixels.

```

1  #define BLUR_SIZE 1
2
3  __global__
4  void blurKernel(const unsigned char *in,
5                  unsigned char *out,
6                  int width,
7                  int height)
8  {
9      int col = blockIdx.x * blockDim.x + threadIdx.x;
10     int row = blockIdx.y * blockDim.y + threadIdx.y;
11
12     if (col < width && row < height) {
13         int sum = 0;
14         int count = 0;
15
16         for (int dy = -BLUR_SIZE; dy <= BLUR_SIZE; ++dy) {
17             for (int dx = -BLUR_SIZE; dx <= BLUR_SIZE; ++dx) {
18                 int r = row + dy;
19                 int c = col + dx;
20
21                 if (r >= 0 && r < height &&
22                     c >= 0 && c < width) {
23                     sum += in[r * width + c];
24                     ++count;

```

```

25         }
26     }
27 }
28
29     out[row * width + col] =
30         static_cast<unsigned char>(sum / count);
31 }
32 }

```

The two key variables are:

- **sum**: accumulated value of all valid neighboring pixels;
- **count**: number of valid pixels actually included in the average.

4.3 Boundary conditions

Pixels near an image boundary do not have a complete 3×3 neighborhood.

For a non-corner edge pixel, six positions are valid. For example, a pixel on the top edge but away from the corners has no row above it, so only two image rows contribute:

$$3 \times 2 = 6$$

valid pixels.

For a corner pixel such as $(0,0)$, only

$$(0,0), (0,1), (1,0), (1,1)$$

are valid, so the average uses only four pixels.

Thus, for a 3×3 blur,

Pixel location	Valid pixels
Interior	9
Non-corner edge	6
Corner	4

This is why the kernel must test every candidate neighbor before reading it, and why the final divisor must be **count** rather than a fixed value such as 9.

5 Matrix Multiplication

5.1 Mathematical operation

For

$$M \in \mathbb{R}^{I \times J}, \quad N \in \mathbb{R}^{J \times K},$$

the product

$$P = MN$$

has shape

$$P \in \mathbb{R}^{I \times K}.$$

Each output element is the dot product of one row of M and one column of N :

$$P_{row,col} = \sum_{k=0}^{J-1} M_{row,k} N_{k,col}$$

For example, if the matrices are 4×4 ,

$$P_{0,0} = M_{0,0}N_{0,0} + M_{0,1}N_{1,0} + M_{0,2}N_{2,0} + M_{0,3}N_{3,0}.$$

The computation of one output element is independent of the computation of the other output elements. Therefore, CUDA can assign one thread to each element of P .

5.2 Mapping one thread to one output element

A two-dimensional CUDA thread computes its output coordinate using

```
1 int row = blockIdx.y * blockDim.y + threadIdx.y;
2 int col = blockIdx.x * blockDim.x + threadIdx.x;
```

The thread at (row, col) computes only

$$P_{row,col}.$$

For square matrices of width `Width`, the core kernel is

```
1 __global__
2 void matrixMulKernel(const float *M,
3                     const float *N,
4                     float *P,
5                     int Width)
6 {
7     int row = blockIdx.y * blockDim.y + threadIdx.y;
8     int col = blockIdx.x * blockDim.x + threadIdx.x;
9
10    if (row < Width && col < Width) {
11        float value = 0.0f;
12
13        for (int k = 0; k < Width; ++k) {
14            value += M[row * Width + k]
15                  * N[k * Width + col];
16        }
17
18        P[row * Width + col] = value;
19    }
20 }
```

The row-major memory accesses are

$$M_{row,k} \longleftrightarrow M[\text{row} * \text{Width} + k],$$

$$N_{k,col} \longleftrightarrow N[k * \text{Width} + \text{col}],$$

and

$$P_{row,col} \longleftrightarrow P[\text{row} * \text{Width} + \text{col}].$$

For the 4×4 example, the thread computing $P_{0,0}$ reads

$$M[0], M[1], M[2], M[3]$$

from the first row of M , and

$$N[0], N[4], N[8], N[12]$$

from the first column of N . It accumulates the four products and finally writes the result to

$$P[0].$$

If a block contains 16×16 threads, that block can compute a 16×16 tile of output elements: each thread produces one element inside the tile. This block-to-output-tile mapping is the basis for later optimized matrix multiplication kernels.

The basic kernel is correct but not optimized. Neighboring threads repeatedly read many of the same matrix elements from global memory, and the accesses down a column of N are strided in row-major storage. Later shared-memory tiling improves data reuse and reduces repeated global-memory traffic.